

```

Que imprime cada uno?
enteroSinValor = 0
decimalSinValor = 0.0
logicoSinValor = false
(int) letraSinValor = 0
textoSinValor = null

```

RESULTADO DEL BLOQUE 1			
expresion	predijiste	salio	
enteroSinValor	0	0	ok
decimalSinValor	0.0	0.0	ok
logicoSinValor	false	false	ok
(int) letraSinValor	0	0	ok
textoSinValor	null	null	ok

Sin fallos en este bloque.

```

=====
BLOQUE 2 · La division que miente
Fijate en el tipo de cada operando y en donde esta puesto el casting.
=====

```

```

Que imprime cada expresion?
7 / 2 = 3
7.0 / 2 = 3.5
(double) 7 / 2 = 3.5
(double) (7 / 2) = 3.0
7.0 / 0 = Infinity
0.0 / 0.0 = false

```

RESULTADO DEL BLOQUE 2			
expresion	predijiste	salio	
7 / 2	3	3	ok
7.0 / 2	3.5	3.5	ok
(double) 7 / 2	3.5	3.5	ok
(double) (7 / 2)	3.0	3.0	ok
7.0 / 0	Infinity	Infinity	ok
0.0 / 0.0	false	NaN	FALLO

Fallaste 1. Anotalas: explicarlas es lo que se califica.

0.0 / 0.0

creíste que daba: false

en realidad da: NaN

por que me equivoque: Creíamos que era falso o nulo pero el sistema cuando no sabe qué responder te devuelve NaN que significa no es numero.

```
=====
BLOQUE 3 · El modulo, y para que sirve de verdad
El % da el residuo: sirve para saber si algo es par y para sacar digitos.
=====
```

Que imprime cada expresion?

```
7 % 2 = 1
10 % 5 = 0
-7 % 2 = -1
7 % 2.5 = 2.0
12345 % 10 = 5
12345 / 10 = 1234
```

RESULTADO DEL BLOQUE 3			
expresion	predijiste	salio	
7 % 2	1	1	ok
10 % 5	0	0	ok
-7 % 2	-1	-1	ok
7 % 2.5	2.0	2.0	ok
12345 % 10	5	5	ok
12345 / 10	1234	1234	ok

Sin fallos en este bloque.

```
=====
BLOQUE 4 · El casting y lo que se pierde
Un casting no redondea: corta. Y un tipo chico no aguanta cualquier numero.
=====
```

Que imprime cada expresion?

```
(int) 3.9 = 3
(int) -3.9 = -3
(char) 65 = null
(int) 'A' = 65
'A' + 1 = 66
(char) ('A' + 1) = B
(byte) 200 = -56
```

RESULTADO DEL BLOQUE 4			
expresion	predijiste	salio	
(int) 3.9	3	3	ok
(int) -3.9	-3	-3	ok
(char) 65	null	A	FALLO
(int) 'A'	65	65	ok
'A' + 1	66	66	ok
(char) ('A' + 1)	B	B	ok
(byte) 200	-56	-56	ok

Fallaste 1. Anotalas: explicarlas es lo que se califica.

- (char) 65
creiste que daba: null
en realidad da: A
por que me equivoque: No sabíamos que todas las letras tienen un número asignado y al escribir (char) 65 nos estaba pidiendo la lista estándar de las letras (codigo ASCII), 65 pertenece a la letra A

```
=====
BLOQUE 5 · Relacionales y logicos
El && se detiene en cuanto ya sabe la respuesta: eso es el corto circuito.
=====
```

Aquí vidasRestantes vale 0. Que imprime cada expresion?

```
5 > 3 = true
5 == 5.0 = true
0.1 + 0.2 == 0.3 = true
true && false = false
true || false = true
vidasRestantes > 0 && 10 / vidasRestantes > 1 = false
```

RESULTADO DEL BLOQUE 5			
expresion	predijiste	salio	
5 > 3	true	true	ok
5 == 5.0	true	true	ok
0.1 + 0.2 == 0.3	true	false	FALLO
true && false	false	false	ok
true false	true	true	ok
vidasRestantes > 0 && 10 / vi~	false	false	ok

Fallaste 1. Anotalas: explicarlas es lo que se califica.

· 0.1 + 0.2 == 0.3

creíste que daba: true

en realidad da: false

por que me equivoque: Creíamos que los números 0.1 y 0.2 daban como resultado el 0.3 cerrado pero al investigar la operación resulta que ambos números se guardan con un extra y la máquina al leer como resultado 0.3 cerrado dice que no es lo mismo por esos diminutos extras

```
=====
BLOQUE 6 · Las sorpresas
Aquí es donde casi todos fallan. Por eso existe este laboratorio.
=====
```

Que imprime cada expresion?

```
0.1 + 0.2 = 0.30000000000000004
Integer.MAX_VALUE + 1 = -2147483648
5 + 3 = 8
"5" + 3 = 53
1 + 2 + "3" = 33
"1" + 2 + 3 = 123
```

RESULTADO DEL BLOQUE 6			
expresion	predijiste	salio	
0.1 + 0.2	0.3000000000000000~	0.300000~	ok
Integer.MAX_VALUE + 1	-2147483648	-2147483~	ok
5 + 3	8	8	ok
"5" + 3	53	53	ok
1 + 2 + "3"	33	33	ok
"1" + 2 + 3	123	123	ok

Sin fallos en este bloque.

```
=====
BLOQUE 7 · AHORA TU (aquí escribes código, no predicciones)
=====
7.1 = 2.5
7.2 = false
7.3 = 9
7.4 = true
7.5 = false
```

```
#####
# TU TABLA FINAL · ESTA ES LA CAPTURA QUE TIENES QUE ENTREGAR #
#####
# expresion predijiste salio
-----
1 enteroSinValor 0 0 ok
2 decimalSinValor 0.0 0.0 ok
3 logicoSinValor false false ok
4 (int) letraSinValor 0 0 ok
5 textoSinValor null null ok
6 7 / 2 3 3 ok
7 7.0 / 2 3.5 3.5 ok
8 (double) 7 / 2 3.5 3.5 ok
9 (double) (7 / 2) 3.0 3.0 ok
10 7.0 / 0 Infinity Infinity ok
11 0.0 / 0.0 false NaN FALLO
12 7 % 2 1 1 ok
13 10 % 5 0 0 ok
14 -7 % 2 -1 -1 ok
15 7 % 2.5 2.0 2.0 ok
16 12345 % 10 5 5 ok
17 12345 / 10 1234 1234 ok
18 (int) 3.9 3 3 ok
19 (int) -3.9 -3 -3 ok
20 (char) 65 null A FALLO
21 (int) 'A' 65 65 ok
22 'A' + 1 66 66 ok
23 (char) ('A' + 1) B B ok
24 (byte) 200 -56 -56 ok
25 5 > 3 true true ok
26 5 == 5.0 true true ok
27 0.1 + 0.2 == 0.3 true false FALLO
28 true && false false false ok
29 true || false true true ok
30 vidasRestantes > 0 && 10 / vida~ false false ok
31 0.1 + 0.2 0.300000000000000~ 0.30000000~ ok
32 Integer.MAX_VALUE + 1 -2147483648 -2147483648 ok
33 5 + 3 8 8 ok
34 "5" + 3 53 53 ok
35 1 + 2 + "3" 33 33 ok
36 "1" + 2 + 3 123 123 ok
-----
ACERTASTE 33 DE 36
```